

Explaining the Spread: Decomposing an FX Quote in kdb+/q

Summary

In production FX pricing systems, a quoted spread is rarely a single, atomic figure — it is the sum of several independently governed adjustments layered on top of a reference level: a base markup, a client-tier skew, a volatility buffer, quote-stability smoothing, a fallback component, and a directional-signal adjustment. That composition isn't a modeling detail; it's operationally load-bearing. Two quotes can land on the same total spread by entirely different routes, and treating the total as one opaque number makes margin attribution, risk review, and client-fairness questions impossible to answer with any precision — you can see *that* pricing moved, not *why*.

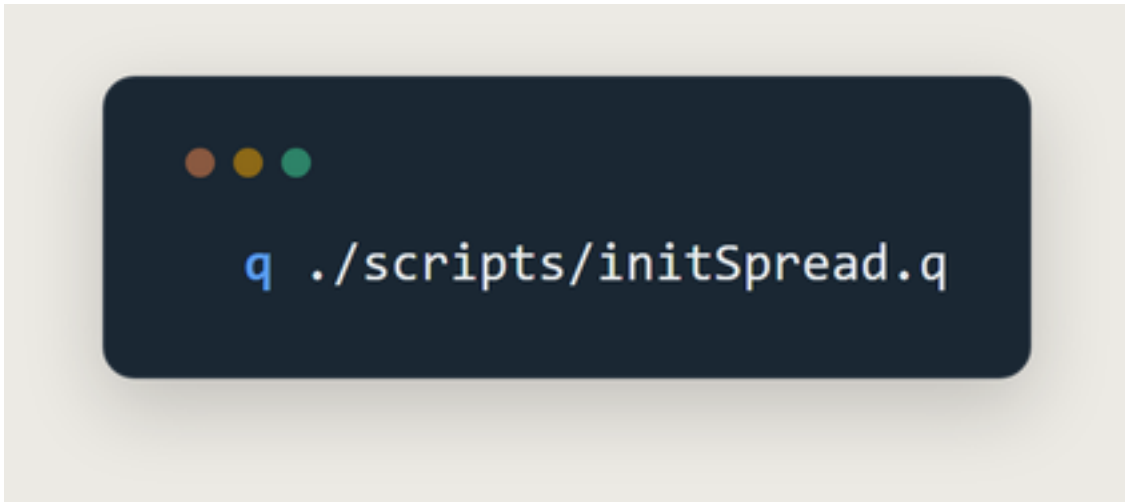
This piece is organized around three questions a pricing or risk team asks of any spread build-up, in increasing order of engineering difficulty:

1. **Composition** — given the named components, what's the total quoted spread? Structurally trivial, a one-line row-wise sum; the risk is entirely in making sure every component is captured once and none is silently dropped or double-counted.
2. **Decomposition and aggregation** — at production scale (thousands of quotes a day across symbols, aggression tiers, and market regimes), how much of the spread is attributable to *which* component, on average, and does that attribution survive being cut by time instead of by tag? This is the question with the most at stake: get the weighting wrong and every downstream rollup misattributes margin or risk to the wrong component, without ever raising an error.
3. **Reconciliation** — is what the model actually quoted consistent with an independent reference — richer, cheaper, in line? This is what turns an internal pricing decomposition into an externally checkable claim, rather than a number the model is left grading on its own homework.

Framed this way, the problem has a different shape from [markout / market impact](#). There, the components are unobserved: a trade produces one noisy price curve, and a temporary/permanent split has to be statistically inferred from it after the fact. Here, the reverse is true — every component is already a column in the row at the moment the quote is generated, so nothing needs to be estimated. The engineering burden moves entirely downstream: aggregating an additive decomposition without corrupting the weighting, and validating the result against a source outside the model itself.

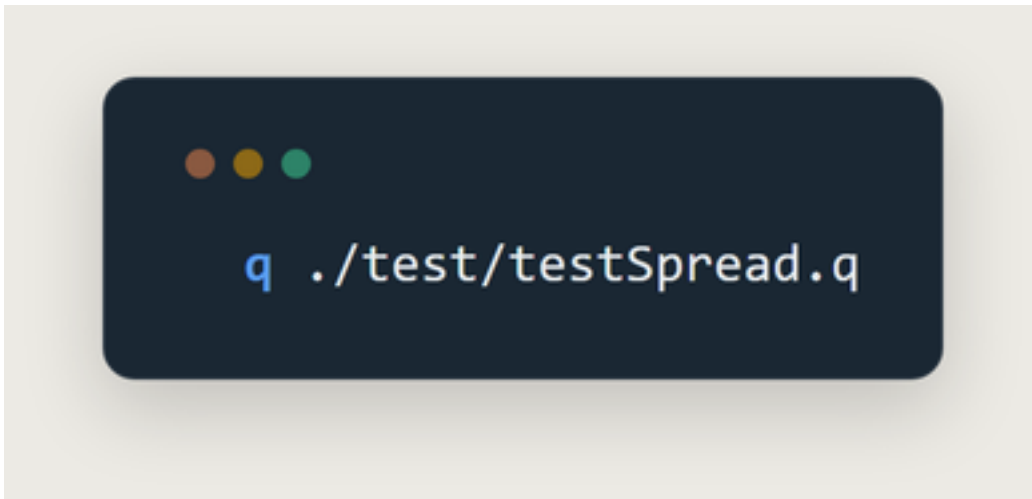
Repo

- The code discussed in this article is at: <https://github.com/SpencerFX/q-link>
- To load the functions, synthetic data, and explore interactively:



q ./scripts/initSpread.q

- To run the test suite (hard assertions, ground-truth checks included):



q ./test/testSpread.q

The component model

The starting assumption is that a quote's total spread decomposes losslessly into seven named, independently sourced components — the pricing engine's own build-up, not a statistical approximation of one reconstructed after the fact. `.spread.componentCols` fixes that vocabulary; `.spread.compose` implements the only claim this section makes: the total is the row-wise sum, nothing more.

```

.spread.componentCols:`anchorSprd`baseSprd`tierSprd`riskSprd`stabilitySprd`fallbackSprd`signalSprd;

.spread.compose:[tab]
  update totalSprd:sum value flip .spread.componentCols#tab from tab
};

```

.spread.componentCols + .spread.compose

Component	What it represents
anchorSprd	reference/baseline spread before any adjustment
baseSprd	core pricing-engine markup
tierSprd	client-tier/relationship skew
riskSprd	volatility risk buffer — widens under elevated vol
stabilitySprd	quote-stability smoothing — dampens jumps between quotes
fallbackSprd	supplemental buffer, used when other inputs are thin
signalSprd	directional-signal adjustment

Component reference table

As an equation: $\text{totalSprd} = \text{anchorSprd} + \text{baseSprd} + \text{tierSprd} + \text{riskSprd} + \text{stabilitySprd} + \text{fallbackSprd} + \text{signalSprd}$ — or more generally, $\text{totalSprd} = \sum \text{over componentCols}$.

```
q)meta scenario`quotes
```

```
c          | t f a
```

```
-----| -----
```

```
time      | p  s
```

```
sym       | s
```

```
aggression | s
```

```
marketStatus | s
```

```
weight    | f
```

```
anchorSprd      | f
```

```
baseSprd      | f
```

```
tierSprd     | f
```

```
riskSprd      | f
```

```
stabilitySprd | f
```

```
fallbackSprd | f
```

```
signalSprd    | f
```

```
totalSprd     | f
```

q)meta scenario`quotes

`.spread.decompose` and `.spread.waterfall` restate that same claim in the two shapes downstream analysis actually needs: `decompose` melts a wide row into one row per (quote, component) — the shape a stacked-bar or attribution view requires — and `waterfall` appends a running cumulative column per component, so `cum_signalSprd == totalSprd` holds on every row by construction. Neither is fitted: there is no residual and no goodness-of-fit statistic to report, because nothing here is being estimated — the exactness itself is the

deliverable.

For each component, `decompose` computes `contributionBps = 1e4 × componentValue`, and `pctOfTotal = 100 × componentValue ÷ totalSprd`. `waterfall`'s cumulative columns follow the same `componentCols` order: `cum_anchorSprd = anchorSprd`, and each subsequent `cum_c = (previous cum_c) + c`, so `cum_signalSprd = totalSprd` by construction — the invariant the test suite asserts directly, rather than trusting that it holds.

One weighting rule, three entry points

Aggregation is where a decomposition like this typically breaks in practice. A single quote's spread carries no information on its own — what a pricing or risk review actually needs is the size-weighted average across many quotes, and that weight has to be applied *identically* regardless of whether the rollup is by regime, by time, or by nothing at all. An inconsistency here doesn't raise an error; it just produces a number that's quietly wrong. The fix applied here is structural rather than procedural: one private helper builds the aggregate-column specification once, and every public rollup is required to route through it.

```
.spread.priv.wavgAggCols: {[wCols]
  (`weight,wCols)!enlist[(sum;`weight)],{(wavg;`weight;x)} each wCols
};

.spread.wavgBy: {[tab;keyCols]
  t:[$(`totalSprd in cols tab;tab;.spread.compose tab];
  wCols:.spread.componentCols,`totalSprd;
  ?[t;());keyCols!keyCols;.spread.priv.wavgAggCols wCols]
};
```

`.spread.priv.wavgAggCols + .spread.wavgBy`

The formula behind every one of those aggregates, for weights w and values x : $\text{wavg} = \Sigma(w \times x) \div \Sigma w$ — exactly what q's built-in `wavg` computes, applied independently to `totalSprd` and each of the seven components rather than to a plain unweighted mean.

`wavgAggCols` builds the spec as a single key-vector/value-vector zip — ``weight,wCols` for keys, `(sum;`weight)` followed by one `(wavg;`weight;col)` tuple per column for values — rather than building a one-item dict per column and unioning them together. The result is identical; the cost isn't: one dict allocation instead of `count[wCols]+1`.

`.spread.byTime` and `.spread.byRegime` are both a handful of lines on top of the same helper: `byTime` swaps in a time-bucket parse-tree as the group-by key, `byRegime` calls `wavgBy` directly with whatever regime columns the caller supplies (e.g. ``aggression`marketStatus`) prepended to any extra keys. The result is one weighting

convention enforced across three distinct entry points, rather than three independent chances to get it wrong.

On data

Validating an aggregation pipeline against real production data has an obvious failure mode: a plausible-looking number and a correct one are indistinguishable without a known answer to check against.

[data/spreadGenerator.q](#) exists to remove that ambiguity — it builds a synthetic session with three effects injected in advance, each carrying a known ground-truth value the analytics functions are then required to recover:

- **A market-status regime shift.** The first half of the session is tagged [normal](#), the second half [stressed](#), and [riskSprd](#) is multiplied by a known factor (**4.0x**) for every stressed quote — nothing else is touched, isolating the effect to one component.
- **An aggression tightening.** [baseSprd](#) and [tierSprd](#) scale by a known per-aggression-level multiplier ([low](#)=1.0, [medium](#)=0.7, [high](#)=0.4), modeling the standard pricing behavior that more aggressive tiers get tighter spreads.
- **An independent benchmark series,** built from the model's own [totalSprd](#) minus a known constant offset (**0.05** price units, i.e. 500 on the [1e4](#)* bps convention [.spread.vsReference](#) uses) plus noise — standing in for a rate the model didn't produce itself, specifically to exercise the reconciliation path.

```
stressFactor:?[marketStatus=`stressed`; .spreadSynth.config.stressVolMult;1f];
riskSprd:0.1*baseLevel*stressFactor*noise[n];
...
benchmark:update benchmarkSprd:totalSprd-richness+0.01*.spreadSynth.priv.randNorm[n]
  from select time,sym,totalSprd from quotes;
```

Injected stress/benchmark snippet

To be clear about what this buys and what it doesn't: this generator is not a model of how a real pricing engine sets a spread. It's a controlled way to know the correct answer in advance — the same role GBM plays in the [markout/impact](#) article's synthetic rate series — so that "the function ran without error" and "the function is correct" remain distinguishable claims.

Interpreting the results

[scripts/initSpread.q](#) runs the recovery check end to end: a 6,000-quote synthetic session (3 symbols, 3 aggression levels, two regimes) through [.spread.wavgBy](#) and [.spread.vsReference](#), leaving [recovery](#) in the workspace as the result — the same assertion [test/testSpread.q](#) runs non-interactively as a pass/fail gate:

```

q)recovery
check          expected recovered relErrPct  pass
-----
stressVolMult  4          4.004076  0.101891   1
richnessBps    500        500.2515  0.05029823 1

```

q)recovery

- **stressVolMult** — grouping all 6,000 quotes by `marketStatus` alone and taking the ratio of average `riskSprd` (stressed ÷ normal) recovers the injected 4.0x multiplier to within ~0.1%.
- **richnessBps** — joining the model's composed `totalSprd` against the independent benchmark series on ``time`sym` via `.spread.vsReference` and averaging the recovered `richnessBps` recovers the injected 500-unit richness to within ~0.05%.

The aggression effect isn't covered by the automated check above, but the same recovery is directly visible in `.spread.byRegime`'s output — comparing `low` against `high` aggression *within the same* `normal` market status:

```

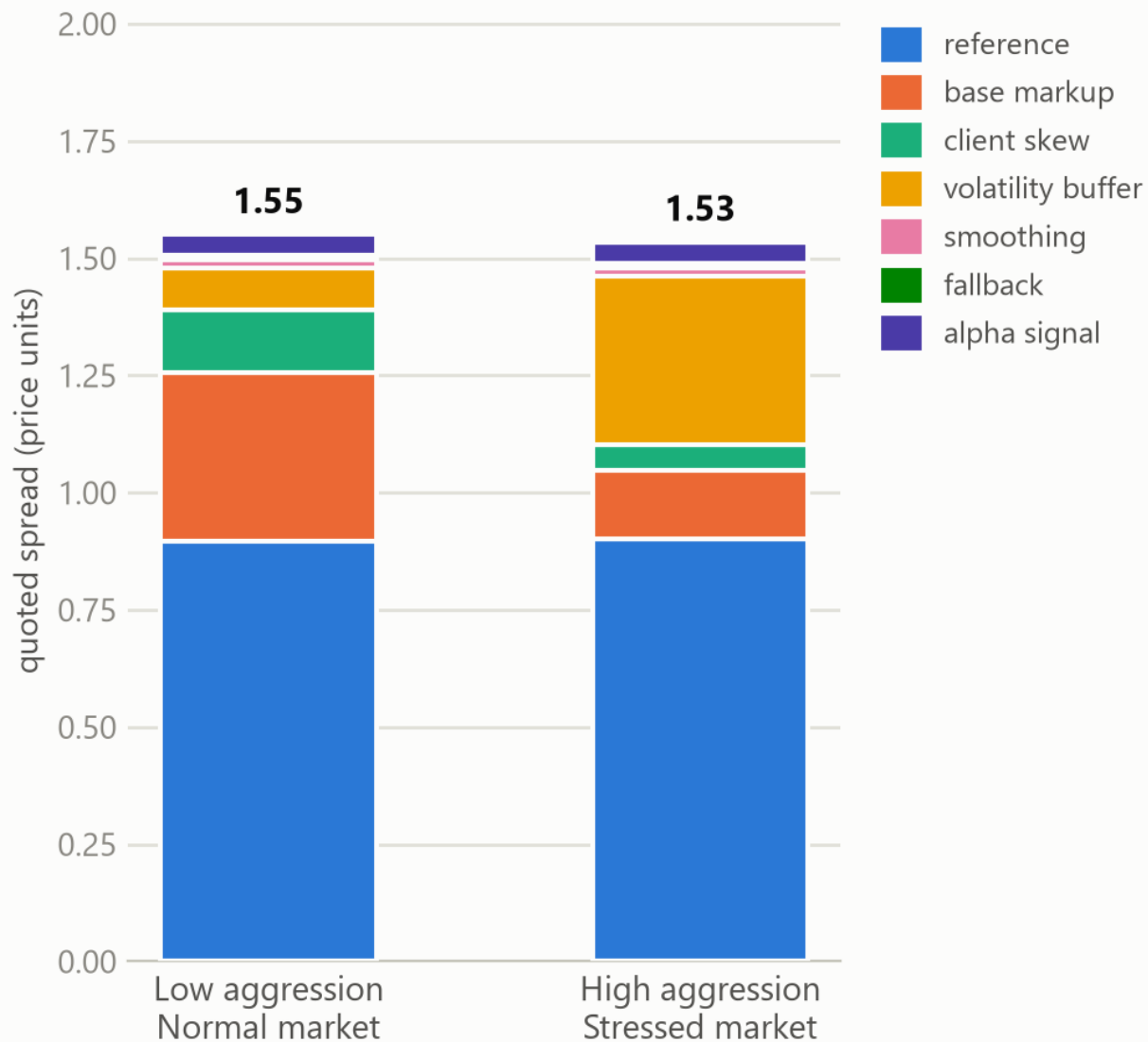
aggression marketStatus  baseSprd  tierSprd
low         normal      0.3585052  0.1343631
high        normal      0.1438458  0.05404622

```

byRegime aggression comparison

$0.1438458 / 0.3585052 \approx 0.401$ and $0.05404622 / 0.1343631 \approx 0.402$ — both independently converge on the injected `aggressionMult[`high]` of 0.4, recovered from two unrelated components without being asked to agree.

Same quote, two regimes: what the spread is actually made of

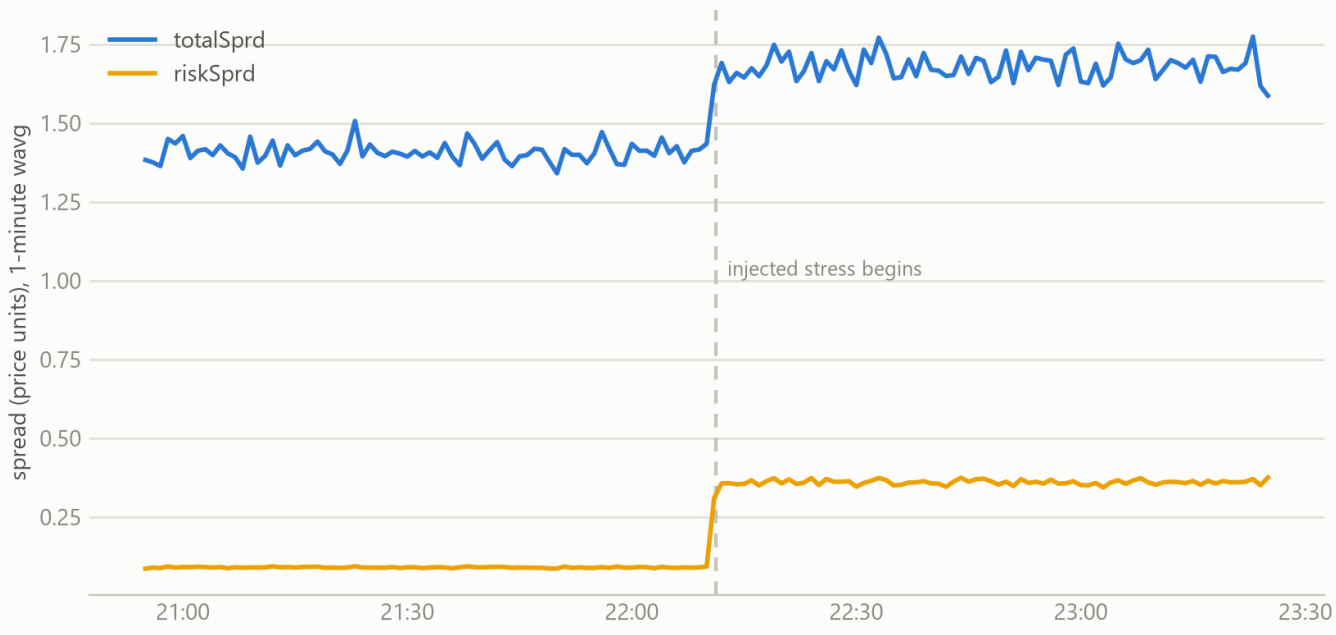


Same quote, priced two ways

That chart deliberately compares two realistic composite scenarios rather than isolating a single variable — a calm, low-aggression quote against an aggressive quote issued into a stressed market — and the result worth noting is what the totals do: **1.55 vs. 1.53, effectively unchanged**. The tighter base markup and client skew from aggressive pricing nearly cancel the wider volatility buffer from the stress regime. A dashboard reporting only `totalSprd` would flag these two quotes as practically identical; the decomposition shows they arrived there by two entirely different routes. That gap — same total, different composition — is the operational case for keeping the components addressable, rather than collapsing them to one number at write time.

Synthetic session · `.spread.byTime[quotes;`minute;`$()]`

riskSprd carries the whole regime shift; the other components barely move



riskSprd carries the regime shift

The second chart rolls the same session up by `.spread.byTime[quotes;`minute;`$()]` instead of by regime tag — a fully independent aggregation path — and recovers the same signal: `riskSprd` jumps at the injected transition, `totalSprd` follows it, and every other component stays flat. Two unrelated rollups agreeing on the same result is itself a form of validation, distinct from either rollup being correct in isolation.

One more composition: share, not just level

`byTime` answers "what's the average level of `riskSprd`" — a related but distinct question is "what *fraction* of the spread is `riskSprd` responsible for, and does that change over the session." Level and share diverge whenever the other components are moving too, and a pricing review that only tracks level can miss a share shift entirely. Answering it required no new machinery: `.spread.decompose` already melts a wide row into one row per component with a `pctOfTotal`, and `.spread.byTime`'s output is quote-shaped by construction (every component column plus `totalSprd`) — so decomposing a `byTime` result instead of raw quotes gives share-over-time as a direct consequence, not a separate feature:

```
.spread.shareByTime: {[tab;bucket;extraKeyCols] .spread.decompose .spread.byTime[tab;bucket;extraKeyCols]};
```

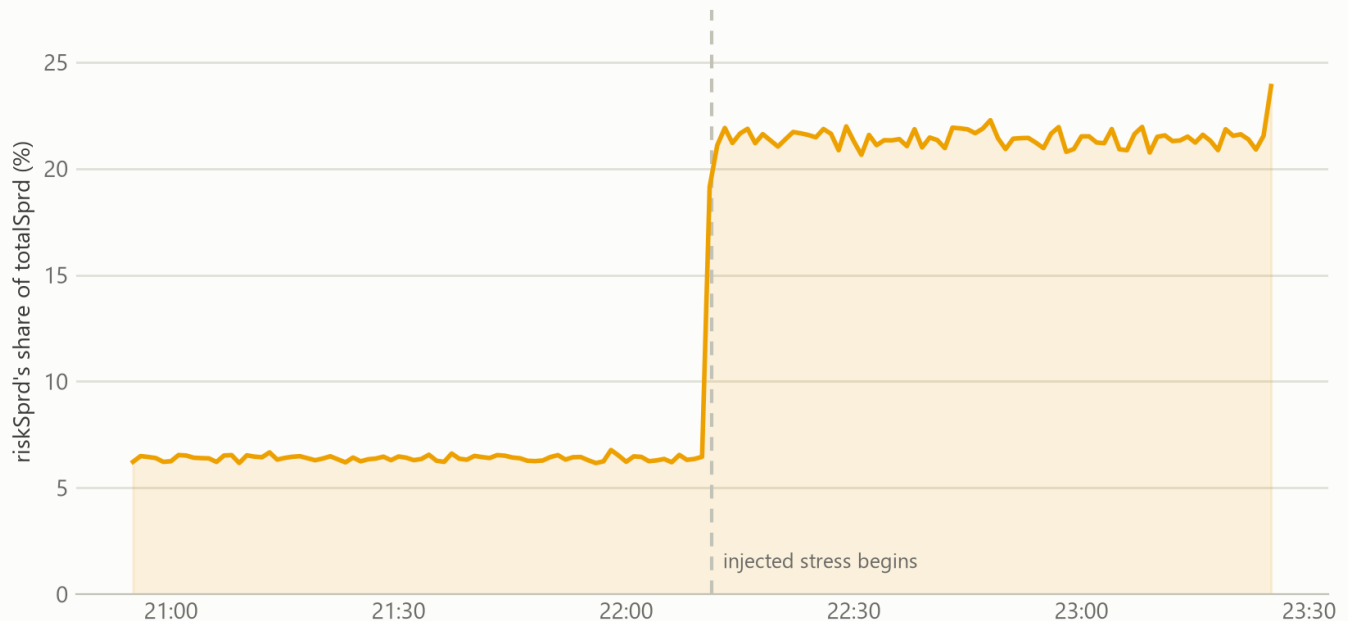
.spread.shareByTime

The order is not incidental. `byTime` has to run *first*: a component's share within a bucket is the ratio of its own weighted average to the bucket's weighted total — not an average of each quote's individual `pctOfTotal`, which produces the wrong answer as soon as weight varies within the bucket. In symbols, per bucket: `pctOfTotal` =

$100 \times \text{wavg}(\text{component}) \div \text{wavg}(\text{totalSprd})$, not the mean of each quote's own $\text{componentValue} \div \text{totalSprd}$. Sequenced this way, `pctOfTotal` sums to exactly 100 within every bucket by construction — the same invariant `.spread.waterfall's cum_signalSprd == totalSprd` provides at the single-row level.

Synthetic session · `.spread.shareByTime[quotes;`minute;`$()]`

Same signal, viewed as a share instead of a level



Same signal, viewed as a share instead of a level

Same session, same transition, same component — but the y-axis now reads "% of `totalSprd`" instead of price units. `riskSprd` moves from ~6% of the quoted spread to ~21% at the stress transition. That framing serves a different reader than the level chart does: a pricing manager asking "is the vol buffer eroding my margin today" needs the share, not the raw number, and the two don't always move together — a component can hold a *constant* share while the total moves, or the reverse.

Making `.spread.decompose` unkey its input defensively (`0!` before the column select) is what makes this composition possible at all. `byTime`'s output is a keyed table — every `?[]` group-by result is — and feeding a keyed table into code written against a plain one is exactly the kind of interface mismatch that looks fine in isolation and only surfaces once two functions are actually chained together.

The mean can hide the tail

Every rollup so far — `wavgBy`, `byTime`, `byRegime`, `shareByTime` — answers with a single weighted average, and an average can look stable while a meaningful part of the underlying distribution is not: three quotes at 1.30 and one at 4.00 average to 1.68, a figure that materially understates what happened on that fourth quote. The question a pricing or risk desk actually needs answered — "how bad does this get" — is a tail question, not a center-of-mass one, and a mean alone cannot answer it.

`wavg` is a q built-in; there's no equivalent built-in for a *weighted* percentile, so `.spread.priv.wpctl` implements the standard nearest-rank method — sort by value, walk cumulative weight in that order, and return the value at the point the cumulative weight fraction first reaches `p`:

```

.spread.priv.wpctl: {[p;w;x]
  ord: iasc x;
  cw: (sums w ord)%sum w;
  (x ord) first where cw>=p
};

```

.spread.priv.wpctl

In symbols: sort x ascending to get order $x(1) \leq x(2) \leq \dots \leq x(n)$, define the cumulative weight fraction $CW(j) = (\sum w(i) \text{ for } i \leq j) \div \sum w$, and return $x(j)$ for the smallest j where $CW(j) \geq p$.

Nearest-rank rather than interpolated is a deliberate choice: the value returned is always an actual observed quote, consistent with how a "worst 1% of the time" figure is conventionally read in a risk context, rather than a smoothed statistical estimate that no single quote ever produced. Because `wpctl` takes the same (weight, value) -> number shape `wavg` does, it drops directly into the same aggregate-spec pattern `.spread.priv.wavgAggCols` already established — `.spread.priv.pctlAggCols` is that function's percentile counterpart, and `.spread.pctlBy/.spread.pctlByTime` are `wavgBy/byTime` with it substituted in:

```

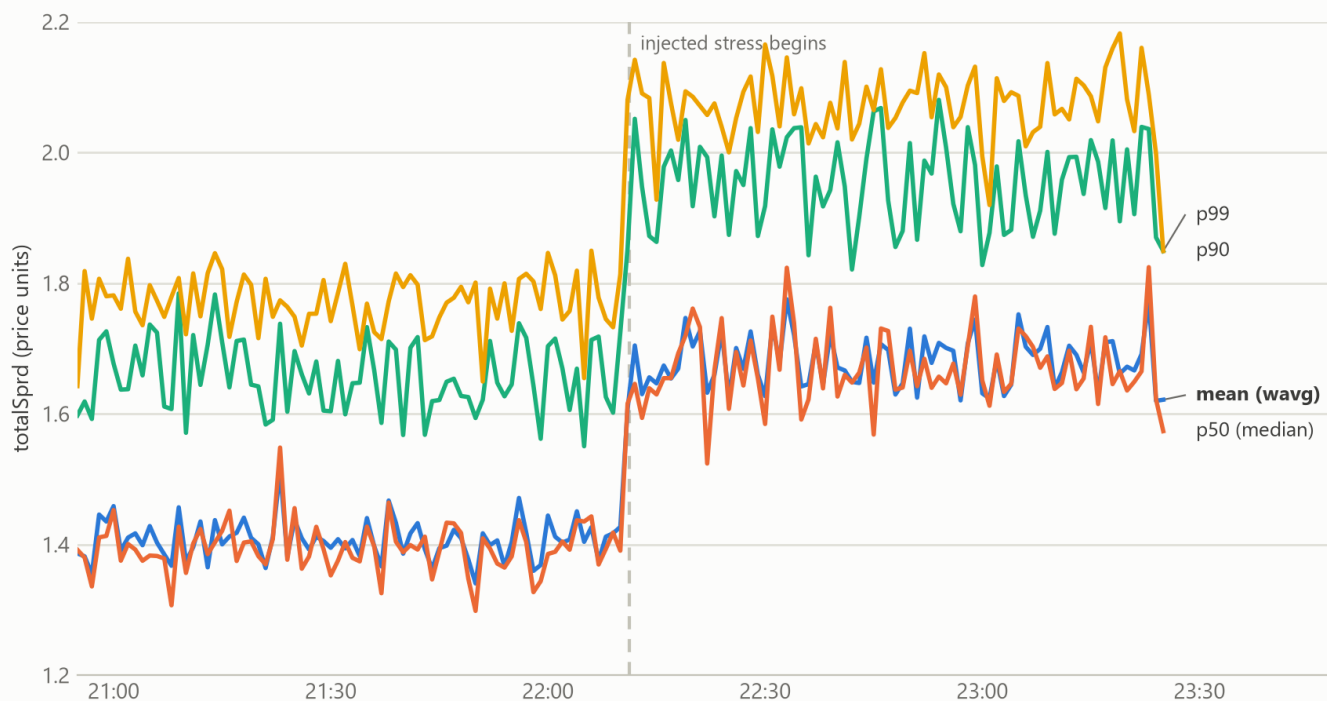
.spread.pctlByTime: {[tab;bucket;extraKeyCols;percentiles]
  t: $[`totalSprd in cols tab;tab;.spread.compose tab];
  aggC: (extraKeyCols!extraKeyCols), enlist[`time]!enlist .spread.util.timeBucket[bucket;`time];
  ?[t;());aggC;.spread.priv.pctlAggCols[percentiles;`totalSprd]]
};

```

.spread.pctlByTime

Synthetic session · `.spread.pctlByTime[quotes;'minute';$();0.5 0.9 0.99]` vs `.spread.byTime`

The whole distribution shifts together, not just the tail



The whole distribution shifts together, not just the tail

Run against the same synthetic session as every other chart in this piece, `p50/p90/p99` all move at the stress transition together with the mean — the gap between them doesn't widen the way it would if the stress were a tail-specific event (an occasional very-wide quote pulling `p99` up while `p50` held steady). That's not an inconclusive result: it *confirms* that the injected stress here is a uniform level-shift affecting every quote equally, exactly matching how `data/spreadGenerator.q` constructed it — `riskSprd`'s multiplier applies to every stressed quote, not to a random subset. Distinguishing these two cases is precisely what percentile analysis is for; this session lands on the "uniform shift" side, and knowing that with certainty is worth more than assuming it from the mean alone.

Sorting isn't free: `perf/perfSpread.q` shows `pctlByTime` at roughly 3.4x `byTime`'s cost (a full sort per bucket vs. one weighted sum) — the one rollup in this library where trading the mean for the distribution has a measurable, quantified price rather than a theoretical one.

Reconciliation vs. an outside reference

`.spread.vsReference` joins the model's composed total against any independently sourced spread series and reports richness in both bps and pct:

```

.spread.vsReference: {[modelTab;refTab;keyCols;refCol]
  m:[$`totalSprd in cols modelTab;modelTab;.spread.compose modelTab];
  mSel:keyCols xkey ?[m;();0b;(keyCols!keyCols),enlist[`modelSprd]!enlist`totalSprd];
  rSel:keyCols xkey ?[refTab;();0b;(keyCols!keyCols),enlist[`benchSprd]!enlist refCol];
  res:0!mSel,'rSel;
  update richnessBps:1e4*modelSprd-benchSprd, richnessPct:100*(modelSprd-benchSprd)%benchSprd from res
};

```

.spread.vsReference

In symbols: $\text{richnessBps} = 1e4 \times (\text{modelSprd} - \text{benchSprd})$, and $\text{richnessPct} = 100 \times (\text{modelSprd} - \text{benchSprd}) \div \text{benchSprd}$.

The immediate use case is a pricing-desk sanity check — is what's being quoted consistent with a competitor feed, a prior model version, or an internal benchmark rate — but the function makes no assumption about what `refTab` actually is. Any two independently produced spread series, reconciled on shared keys, fit the same shape: a required check, not a bespoke one built per comparison.

Conclusions

Spread decomposition and market impact sit under the same broad heading — pricing and post-trade analytics — while being close to opposite problems in shape. Market impact starts from one noisy observable (price after the fact) and has to recover a hidden temporary/permanent structure from it, which is why that analysis required a parametric fit and a goodness-of-fit statistic before its recovery could be trusted. Spread decomposition starts from the opposite position: the structure is already given, every component is a column the pricing engine already computed, and there is nothing left to estimate. The entire engineering problem is downstream of that — aggregating an additive decomposition without corrupting the weighting regardless of which dimension it's sliced by, and validating the result against something the model itself did not produce.

Both pieces are held to the same standard for knowing the code is correct rather than merely plausible: construct a synthetic scenario with a known answer built in, and require the functions to reproduce that specific number — not something in the right ballpark.

Appendix: performance

Correctness established, the remaining question is cost: what this decomposition- and-aggregation pipeline actually takes to run at production-representative volumes. `perf/perfChk.q` is a shared, functions-only timing harness; per-article runners load it and time every public function in their own analytics file against realistic-sized synthetic data, via `kdb+`'s built-in `\ts` time+space profiler (invoked programmatically as `system"ts do[n;expr]"`, so the (ms;bytes) pair is captured and averaged over `n` reps rather than merely printed to console). The table below covers the `.util.*` and `.spread.*` subset — the shared offset-grid helpers plus every function discussed in this piece — run at 5x the session size used above: a 216,000-row rate series, 10,000 trades, 25 orders, and 30,000 quotes.

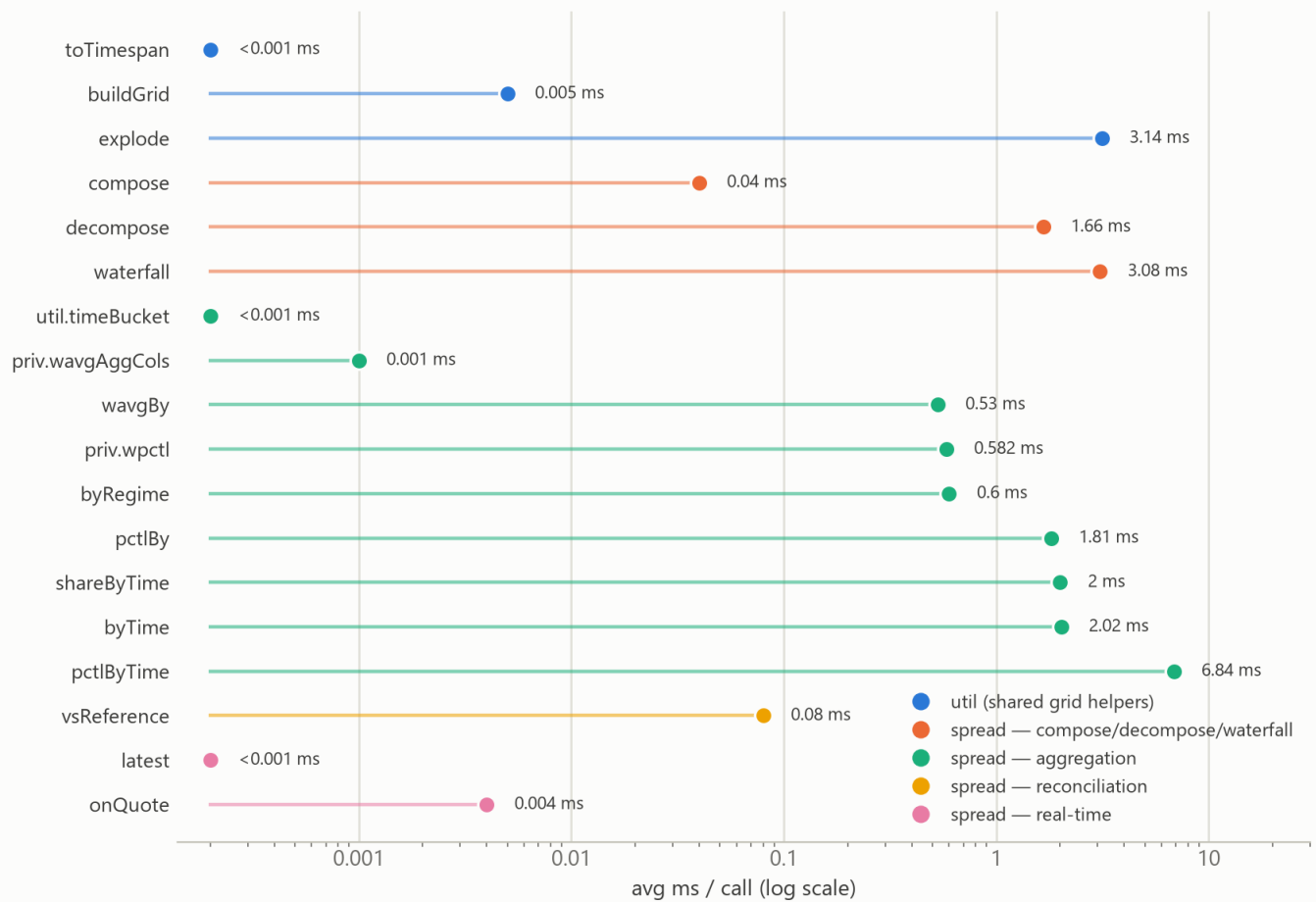


```
q perf/perfMarkOut.q # .util.* (this table's util rows)
q perf/perfSpread.q   # .spread.* (this table's spread rows)
```

perf runner commands

perf/perfChk.q · 216,000-row rate series / 10,000 trades / 25 orders / 30,000 quotes

Every .util.* and .spread.* function, timed at 5x scale



Every .util. and .spread.* function, timed at 5x scale*

Function	Reps	Avg ms/call
<code>.util.buildGrid</code>	2000	0.005
<code>.util.toTimespan</code>	2000	<0.001
<code>.util.explode</code>	50	3.14
<code>.spread.compose</code>	100	0.05
<code>.spread.decompose</code>	50	1.52
<code>.spread.waterfall</code>	50	4.04
<code>.spread.priv.wavgAggCols</code>	2000	0.001
<code>.spread.util.timeBucket</code>	2000	<0.001
<code>.spread.wavgBy</code>	100	0.47
<code>.spread.byRegime</code>	100	0.59
<code>.spread.byTime</code>	50	1.98
<code>.spread.shareByTime</code>	50	1.9
<code>.spread.priv.wpctl</code>	2000	0.582
<code>.spread.pctlBy</code>	100	1.81
<code>.spread.pctlByTime</code>	50	6.84
<code>.spread.vsReference</code>	50	0.08
<code>.spread.onQuote</code>	1000	0.004
<code>.spread.latest</code>	1000	<0.001

Performance results table

A few things stand out. The pure grid/dict helpers (`buildGrid`, `toTimespan`, `priv.wavgAggCols`, `util.timeBucket`) are all sub-microsecond to low-microsecond — they build small, fixed-size structures and never touch the quote table. `.spread.compose` is close to free (0.05ms for 30,000 rows), being a single row-wise sum; `decompose` and `waterfall` cost more (1.5–4ms) because each builds several full-sized intermediate tables — one per component, or one cumulative column per component — rather than a single pass. The aggregations

([wavgBy](#), [byRegime](#), [byTime](#)) stay under 2ms even while grouping and weight-averaging all 30,000 rows; [byTime](#) is the most expensive of the three because its time-bucket key produces more distinct groups than a coarse regime tag does. [shareByTime](#) costs almost exactly what [byTime](#) alone does (1.9ms vs. 1.98ms), since [decompose](#) only runs against the small, already-aggregated bucket table it produces rather than the original 30,000 rows — melting it into shares is close to free on top. The percentile rollups are the clear outliers: [pctlBy](#) costs roughly 3.8x [wavgBy](#)'s (1.81ms vs. 0.53ms), and [pctlByTime](#) costs roughly 3.4x [byTime](#)'s (6.84ms vs. 2.02ms) — the one place in this library where a full sort per group, rather than a running sum, materially shows up in the numbers. [.spread.onQuote](#) (the real-time path) is flat at 0.004ms regardless of session size, as required — it upserts one row into a table keyed by a small, bounded (sym, aggression, marketStatus) key space, never touching the historical quote volume.

The two `<0.001` rows ([toTimespan](#), [latest](#)) reported exactly 0 from the profiler — below `\ts`'s millisecond resolution at this call cost, not a claim that the operation is literally free.